Text Search 文本搜索

Qdrant is a vector search engine, making it a great tool for semantic search. However, Qdrant's capabilities go beyond just vector search. It also supports a range of lexical search features, including filtering on text fields and full-text search using popular algorithms like BM25.

Qdrant 是一款矢量搜索引擎，是语义搜索的优秀工具。然而，Qdrant 的功能不仅限于矢量搜索。它还支持多种词汇搜索功能，包括文本字段过滤和使用流行算法如 BM25 进行全文搜索。

Semantic Search 语义搜索

Semantic search is a search technique that focuses on the meaning of the text rather than just matching on keywords. This is achieved by converting text into vectors (embeddings) using machine learning models. These vectors capture the semantic meaning of the text, enabling you to find similar text even if it doesn't share exact keywords.

语义搜索是一种专注于文本意义的搜索技术，而不仅仅是关键词匹配。这是通过利用机器学习模型将文本转换为向量 （嵌入）来实现的。这些向量捕捉文本的语义意义，使你能够找到相似文本，即使它们不完全共享关键词。

> ⓘ The examples in this guide use inference to let Qdrant generate the vectors. Inference is only available on Qdrant Cloud, with the exception of the BM25 model. If you are not running on Qdrant Cloud, you can use a library like FastEmbed to generate vectors on the client side. When using FastEmbed, refer to the documentation, as its API may differ from that of server-side inference.
>
> 本指南中的示例使用推理让 Qdrant 生成向量。推理仅在 Qdrant Cloud 上可用，BM25 模型除外。如果你不在 Qdrant Cloud 上运行，可以使用像 FastEmbed 这样的库在客户端生成向量。使用 FastEmbed 时，请参考文档，因为其 API 可能与服务器端推理不同。

For example, to search through a collection of books, you could use a model like the `all-MiniLM-L6-v2` sentence transformer model. First, create a collection and configure a dense vector for the book descriptions:

例如，要搜索一堆书籍，你可以使用 像 `all-MiniLM-L6-v2` 句子变换器模型这样的模型。首先，创建一个集合，并为书籍描述配置一个密集向量：

```
PUT /collections/books
{
  "vectors": {
    "description-dense": {
      "size": 384,
      "distance": "Cosine"
    }
  }
}
```

Next, you can ingest data:

接下来，你可以导入数据：

```
PUT /collections/books/points?wait=true
{
  "points": [
    {
      "id": 1,
      "vector": {
        "description-dense": {
          "text": "A Victorian scientist builds a device to travel far
          "model": "sentence-transformers/all-minilm-l6-v2"
        }
      },
      "payload": {
        "title": "The Time Machine",
        "author": "H.G. Wells",
        "isbn": "9780553213515"
      }
    }
  ]
}
```

To find books related to “time travel”, use the following query:

要查找与“时间旅行”相关的书籍，请使用以下查询：

```
POST /collections/books/points/query
{
  "query": {
    "text": "time travel",
    "model": "sentence-transformers/all-minilm-l6-v2"
  },
  "using": "description-dense",
  "with_payload": true
}
```

In these examples, Qdrant uses inference to generate vectors from the `text` provided in the request using the specified `model`. Alternatively, you can generate explicit vectors on the client side with a library like FastEmbed.

在这些例子中，Qdrant 利用推理从请求中提供的 `文本` 生成向量，并结合指定的 `模型` 。或者，你也可以在客户端用像 FastEmbed 这样的库生成显式向量。

Lexical Search 词汇检索

Lexical search, also known as keyword-based search, is a traditional search technique that relies on matching words or phrases in the text. Many applications require a combination of semantic and traditional lexical search. A good example is in e-commerce, where users may want to search for products using a product ID. ID values don't lend themselves well to vectorization, but being able to search for them is essential for a good search experience. To facilitate these use cases, Qdrant enables you to use lexical search alongside semantic search.

词汇搜索，也称为基于关键词的搜索，是一种依赖文本中匹配词语或短语的传统搜索技术。许多应用需要语义搜索和传统词汇搜索的结合。一个很好的例子是电子商务，用户可能想用产品 ID 搜索产品。ID 值不太适合向量化，但能够搜索它们对于良好的搜索体验至关重要。为了促进这些用例，Qdrant 允许你将词汇搜索与语义搜索结合使用。

Filtering Versus Querying 筛选与查询

When it comes to lexical search in Qdrant, it's important to distinguish between filtering and querying. Filtering is used to narrow down results based on exact matches or specific criteria, while querying involves finding relevant documents based on the content of the text. In other words, filtering is about precision, while querying is about recall. A filter does not contribute to the ranking of search results, as no score is calculated for filters. A query calculates a

relevance score for each matching document and that score is used to rank search results.

在 Qdrant 中，区分筛选和查询非常重要。过滤用于根据精确匹配或特定条件缩小结果范围，而查询则是根据文本内容寻找相关文档。换句话说，过滤注重准确性，查询则注重回忆。过滤器不对搜索结果排名有贡献，因为筛选不计算分数。查询会为每个匹配文档计算相关性评分，并用该分数对搜索结果进行排名。

Filter 滤波器	Query 查询
Does not contribute to ranking 不会影响排名	Contributes to ranking 对排名的贡献
Improves precision by narrowing down results 通过缩小结果范围提升精度	Improves recall by finding relevant data 通过寻找相关数据提升回忆能力

Filtering 过滤

Qdrant supports filtering on a wide range of datatypes: numbers, dates, booleans, geolocations, and strings. In Qdrant, a filter is typically combined with a vector query. The vector query is used to score and rank the results, while the filter is used to narrow down the results based on specific criteria.

Qdrant 支持对多种数据类型的过滤：数字、日期、布尔值、地理位置和字符串。在 Qdrant 中，滤波器通常与向量查询结合使用。向量查询用于对结果进行评分和排序，而过滤器则用于根据特定标准缩小结果范围。

Text and Keyword Strings 文本和关键词字符串

When it comes to filtering on strings, it is important to understand the difference between the two types of strings in Qdrant: text and keyword. These two string types are designed for different use cases: filtering on exact string values or filtering on individual search terms. To filter on exact string values, Qdrant uses **keyword** strings. Keyword strings are ideal for filtering on strings like IDs, categories, or tags. To filter on individual terms or phrases within a larger body of text, Qdrant uses **text** strings.

在字符串过滤时，理解 Qdrant 中两种字符串的区别非常重要：文本字符串和关键词。这两种字符串类型设计用于不同的用例：针对精确字符串值过滤，或针对单个搜索词进行筛选。为了筛选精确字符串值，Qdrant 使用**关键词**字符串。关键词串非常适合用于对 ID、类别或标签等字符串进行过滤。为了在更大文本中筛选单个术语或短语，Qdrant 使用**文本**字符串。

For example, take a string like “United States”. If you want to filter on all points with this exact string in the payload, use a keyword filter. On the other hand, if you want to filter on all points

that contain the word “united” (matching “United States” as well as “United Kingdom”), use a text filter.

例如，拿字符串“United States”来说。如果你想对所有带有该字符串的点进行筛选，可以使用关键词过滤器。另一方面，如果你想筛选所有包含“联合”一词的点（同时匹配“美国”和“联合王国”），可以使用文本过滤器。

Keyword 关键词	Text 正文
Used for exact string matches 用于精确的弦匹配	Used for filtering on individual terms 用于单个术语的过滤
Ideal for IDs, categories, tags 非常适合 ID、类别、标签	Ideal for larger text fields 适合较大的文本字段
Not tokenized 未代币化	Tokenized into individual terms 被标记为单独的术语
Case-sensitive 大小写区分	Case-insensitive by default 默认情况下不区分大小写

Filtering on an Exact String
在精确字符串上的滤波

To filter on exact strings, first create a payload index of type `keyword` for the field you want to filter on. A payload index makes filtering faster and reduces the load on the system.

要在精确字符串上进行筛选，首先为你想过滤的字段创建一个类型 为关键字的 有效载荷索引 。有效载荷指数使过滤更快，减轻系统负载。

> ⓘ Filtering on a field without an index is not possible on collections that run in strict mode. Strict mode is enabled by default on Qdrant Cloud.
>
> 在严格模式下运行的集合中，无法对无索引字段进行过滤。严格模式在 Qdrant Cloud 默认是开启的。

For example, to filter books by author name, create a keyword index on the “author” field:

例如，要按作者名筛选书籍，可以在“作者”字段创建关键词索引：

```
PUT /collections/books/index?wait=true
{
    "field_name": "author",
    "field_schema": "keyword"
}
```

Next, when querying the data, you also add a filter clause to the request. The following example searches for books related to “time travel” but only returns books written by H.G. Wells:

接下来，在查询数据时，你还要在请求中添加一个过滤子句。以下示例搜索与“时间旅行”相关的书籍，但只返回 H.G.威尔斯所著的书籍：

```
POST /collections/books/points/query
{
  "query": {
    "text": "time travel",
    "model": "sentence-transformers/all-minilm-l6-v2"
  },
  "using": "description-dense",
  "with_payload": true,
  "filter": {
    "must": [
      {
        "key": "author",
        "match": {
          "value": "H.G. Wells"
        }
      }
    ]
  }
}
```

The ranking of the results of this request is based on the vector similarity of the query. The filter only narrows down the results to those points where the `author` field exactly matches `H.G. Wells`. Furthermore, the filter is case-sensitive. Filtering for the lowercase value `h.g.`

`wells` would not return any results.

该请求结果的排名基于查询的向量相似度。筛选器只将结果缩小到 `作者字段与 H.G.威尔斯` 完全匹配的点。此外，滤波器是区分大小写的。过滤小写值 `h.g.阱` 不会返回任何结果。

The previous example only returns points that match the filter value. If you want the opposite: exclude points with a specific value, use a `must_not` clause instead of `must`. The following example only returns books *not* written by H.G. Wells:

前例只返回与滤波器值匹配的点。如果你想要相反的：排除具有特定分值的分数，可以用 `must_not` 子句代替 `必须` 。以下示例仅返回*非* H.G.威尔斯所著书籍：

```
POST /collections/books/points/query
{
  "query": {
    "text": "time travel",
    "model": "sentence-transformers/all-minilm-l6-v2"
  },
  "using": "description-dense",
  "with_payload": true,
  "filter": {
    "must_not": [
      {
        "key": "author",
        "match": {
          "value": "H.G. Wells"
        }
      }
    ]
  }
}
```

Filtering on Multiple Exact Strings
多条精确字符串的滤波

You can provide multiple filter clauses. For example, to find all books co-authored by Larry Niven and Jerry Pournelle, use the following filter:

你可以提供多个过滤条款。例如，要查找拉里·尼文和杰瑞·波内尔合著的所有书籍，请使用以下筛选器：

```
POST /collections/books/points/query
{
  "query": {
    "text": "space opera",
    "model": "sentence-transformers/all-minilm-l6-v2"
  },
  "using": "description-dense",
  "with_payload": true,
  "filter": {
    "must": [
      {
        "key": "author",
        "match": {
          "value": "Larry Niven"
        }
      },
      {
        "key": "author",
        "match": {
          "value": "Jerry Pournelle"
        }
      }
    ]
  }
}
```

Note that both filter clauses must be true for a point to be included in the results, because a `must` clause operates like a logical `AND`. If you want to find books written by either author (as well as both), use a `should` clause, which operates like a logical `OR`:

注意，两个滤波子句都必须为真，才能包含某个点，因为 必须 子句的作用类似于逻辑 `AND`。如果你想找到任一作者（以及两者）写的书，可以使用 应从 句，这就像逻辑上的 `OR`：

```
POST /collections/books/points/query
{
  "query": {
    "text": "space opera",
    "model": "sentence-transformers/all-minilm-l6-v2"
  },
  "using": "description-dense",
```

```
    "with_payload": true,
    "filter": {
      "should": [
        {
          "key": "author",
          "match": {
            "value": "Larry Niven"
          }
        },
        {
          "key": "author",
          "match": {
            "value": "Jerry Pournelle"
          }
        }
      ]
    }
  }
```

Alternatively, when you want to filter on one or more values of a single key, you can use the `any` condition:

或者，当你想对单个键的一个或多个值进行过滤时，可以使用 `任意` 条件：

```
POST /collections/books/points/query
{
  "query": {
    "text": "space opera",
    "model": "sentence-transformers/all-minilm-l6-v2"
  },
  "using": "description-dense",
  "with_payload": true,
  "filter": {
    "must": [
      {
        "key": "author",
        "match": {
          "any": ["Larry Niven", "Jerry Pournelle"]
        }
      }
    ]
  }
```

```
}
```

Full-Text Filtering 全文过滤

In contrast to keyword filtering, filtering on text strings enables you to filter on individual words within a larger text field in a case-insensitive manner. To understand how this works, it's important to understand how text strings are processed at index time and query time.

与关键词过滤不同，文本字符串过滤允许你以不区分大小写的方式过滤更大文本字段中的单个单词。要理解其工作原理，重要的是了解文本字符串在索引和查询时的处理方式。

Text Processing 文本处理

To enable efficient full-text filtering, Qdrant processes text strings by breaking them down into individual tokens (words) and applying several normalization steps. This process ensures that searches are more flexible and can match variations of words. At query time, Qdrant applies the same processing steps to the filter string, ensuring that the filter matches the indexed tokens correctly.

为了实现高效的全文过滤，Qdrant 通过将文本字符串拆分为单个词（单词）并应用若干规范化步骤来处理。这一过程确保搜索更灵活，能够匹配词汇的变体。在查询时，Qdrant 对过滤字符串应用相同的处理步骤，确保过滤器正确匹配索引标记。

The following text processing steps are applied to text strings:

以下文本处理步骤应用于文本字符串：

- The string is broken down into individual tokens (words) using a process called tokenization. By default, Qdrant uses the `word` tokenizer, which splits the string using word boundaries, discarding spaces, punctuation marks, and special characters.

  字符串通过称为分词化的过程被拆分成单个词（单词）。默认情况下，Qdrant 使用 `单词` 分词器，通过单词边界、丢弃空格、标点符号和特殊字符来拆分字符串。

- By default, each word is then converted to lowercase. Lowercasing the tokens allows Qdrant to ignore capitalization, making full-text filters case-insensitive.

默认情况下，每个单词随后会转换为小写 。缩小符号可以让 Qdrant 忽略大小写，使全文过滤器不区分大小写。

- Optionally, Qdrant can remove diacritics (accents) from characters using a process called ASCII folding. This ensures that diacritics are ignored. As a result, filtering for the word “cafe” matches “café”.
- Optionally, tokens can be reduced to their root form using a stemmer. This ensures that filtering for “running” also matches “run” and “ran”. Because stemming is language-specific, if enabled, it must be configured for a specific language.
- Certain words like “the”, “is”, and “and” are very common in text and do not contribute much to the meaning of text. These words are called stopwords and can optionally be removed during indexing. Like stemming, stopword removal is language-specific. You can configure specific languages for stopword removal and/or provide a custom list of stopwords to remove.
- Optionally, you can enable phrase matching to allow filtering for multiple words in the exact same order as they appear in the original text.

These text processing steps can be configured when creating a full-text index. For example, to create a text index on the `title` field with ASCII folding enabled:

```
PUT /collections/books/index?wait=true
{
  "field_name": "title",
  "field_schema": {
    "type": "text",
    "ascii_folding": true
  }
}
```

When querying using this index, Qdrant automatically applies the same text processing steps to the filter string before matching it against the indexed tokens.

Filter on Text Strings

To filter on text values in a payload field, first create a full-text index for that field. Next, you can use a `text` condition to query the collection with a filter for titles that contain the word “space”:

```
POST /collections/books/points/query
```

```
{
  "query": {
    "text": "space opera",
    "model": "sentence-transformers/all-minilm-l6-v2"
  },
  "using": "description-dense",
  "with_payload": true,
  "filter": {
    "must": [
      {
        "key": "title",
        "match": {
          "text": "space"
        }
      }
    ]
  }
}
```

When filtering on more than one term, a `text` filter only matches fields that contain *all* the specified terms (logical `AND`). To match fields that contain *any* of the specified terms (logical `OR`), use the `text_any` condition:

```
POST /collections/books/points/query
{
  "query": {
    "text": "space opera",
    "model": "sentence-transformers/all-minilm-l6-v2"
  },
  "using": "description-dense",
  "with_payload": true,
  "filter": {
    "must": [
      {
        "key": "title",
        "match": {
          "text_any": "space war"
        }
      }
    ]
  }
```

```
}
```

Qdrant also supports phrase filtering, enabling you to search for multiple words in the exact order they appear in the original text, with no other words in between. For example, a phrase filter for “time machine” matches against the title “The Time Machine” but would not match “The Time Travel Machine” (there’s a word between “time” and “machine”) nor “Machine Time” (the word order is incorrect).

The difference between phrase filtering and keyword filtering is that phrase filtering applies text processing and, as a result, is case-insensitive, while keyword filtering is case-sensitive and only matches the exact string. Additionally, keyword filtering has to match the entire string, whereas phrase filtering can match part of a larger string. So a keyword filter for “Space War” would not match “The Space War” because it doesn’t match “The,” but a phrase filter for “Space War” would.

Summarizing the differences between the four filtering methods for a multi-term filter on “Space War”:

Method	Actual query	Matches `Space War`?	Matches `The Space War`?	Matches `War in Space`?	Matches `War of the Worlds`?
text_any	`space OR war`	Yes	Yes	Yes	Yes
text	`space AND war`	Yes	Yes	Yes	No
phrase	`"space war"`	Yes	Yes	No	No
keyword	`"Space War"`	Yes	No	No	No

To filter on phrases, use a `phrase` condition. This requires enabling phrase searching when creating the full-text index:

```
PUT /collections/books/index?wait=true
{
  "field_name": "title",
  "field_schema": {
    "type": "text",
    "ascii_folding": true,
    "phrase_matching": true
  }
```

```
}
```

Next, you can use a `phrase` condition to filter for titles that contain the exact phrase “time machine”:

```
POST /collections/books/points/query?wait=true
{
  "query": {
    "text": "time travel",
    "model": "sentence-transformers/all-minilm-l6-v2"
  },
  "using": "description-dense",
  "with_payload": true,
  "filter": {
    "must": [
      {
        "key": "title",
        "match": {
          "phrase": "time machine"
        }
      }
    ]
  }
}
```

Progressive Filtering with the Batch Search API

Even though filters are not used to rank results, you can use the batch search API to progressively relax filters. This is useful when you have strict filtering criteria that may not return results. Batching multiple search requests with progressively relaxed filters enables you to get results even when the strictest filter returns no results.

For example, the following batch search request first tries to find books that match all search terms in the title. The second search request relaxes the filter to match any of the search terms. The third search request removes the filter altogether:

```
POST /collections/books/points/query/batch
{
  "searches": [
```

```
{
  "query": {
    "text": "time travel",
    "model": "sentence-transformers/all-minilm-l6-v2"
  },
  "using": "description-dense",
  "with_payload": true,
  "filter": {
    "must": [
      {
        "key": "title",
        "match": {
          "text": "time travel"
        }
      }
    ]
  }
},
{
  "query": {
    "text": "time travel",
    "model": "sentence-transformers/all-minilm-l6-v2"
  },
  "using": "description-dense",
  "with_payload": true,
  "filter": {
    "must": [
      {
        "key": "title",
        "match": {
          "text_any": "time travel"
        }
      }
    ]
```

```
{
  "query": {
    "text": "time travel",
    "model": "sentence-transformers/all-minilm-l6-v2"
  },
  "using": "description-dense",
  "with_payload": true
}
```

```
        ]
    }
```

The response contains three separate result sets. You can return the first non-empty result set to the user, or you can use the three sets to assemble a single ranked list.

Full-Text Search

Full-text search is similar to full-text filtering, with the key difference being that full-text queries are used for ranking. For each document that matches the search terms, Qdrant calculates a relevance score based on how well the document matches the search terms. That score is used to rank the results. Qdrant supports several full-text search scoring algorithms.

Full-text search in Qdrant is powered by sparse vectors. Why sparse vectors? Because they are a flexible way to represent data for search purposes, from classic BM25-based search, to semantic search, and collaborative filtering. Each term in the vocabulary corresponds to one or more dimension of the sparse vector, and the values in those dimensions represent the weight of that term in the document. Weights can be calculated using document statistics for use with the BM25 ranking algorithm, or you can use transformer-based models that can capture semantic meaning, like SPLADE++, and miniCOIL.

BM25

BM25 (Best Matching 25) is a popular ranking algorithm that takes a probabilistic approach to score calculation. For each search term, BM25 considers several statistics about the term and the document to calculate a relevance score:

- Term frequency (TF): the more often a term appears in a document, the more relevant that document is likely to be.
- Inverse document frequency (IDF): the rarer a term is across all documents, the higher the weight of that term.
- Document length: a term appearing in a shorter document is more relevant than the same term appearing in a longer document.

Qdrant provides native support for BM25 through an inference model that generates sparse vectors, or you can generate vectors on the client side using the FastEmbed library.

The BM25 model supports the same text processing options as text indices, including tokenization, lowercasing, ASCII folding, stemming, and stopword removal. A notable difference with text indices is that BM25 defaults to English stemming and stopword removal. If you are using a language other than English, ensure that you configure the model accordingly.

To use BM25, configure a sparse vector when creating a collection:

```
PUT /collections/books?wait=true
{
  "sparse_vectors": {
    "title-bm25": {
      "modifier": "idf"
    }
  }
}
```

Note the IDF modifier, which configures the sparse vector for queries that use the inverse document frequency (IDF).

Now you can ingest data. The following example ingests a book with its title represented as a sparse vector generated by the BM25 model:

```
PUT /collections/books/points?wait=true
{
  "points": [
    {
      "id": 1,
      "vector": {
        "title-bm25": {
          "text": "The Time Machine",
          "model": "qdrant/bm25"
        }
      },
      "payload": {
        "title": "The Time Machine",
        "author": "H.G. Wells",
        "isbn": "9780553213515"
      }
    }
  ]
}
```

After ingesting data, you can query the sparse vector. The following example searches for books with “time travel” in the title using the BM25 model:

```
POST /collections/books/points/query
{
  "query": {
    "text": "time travel",
    "model": "qdrant/bm25"
  },
  "using": "title-bm25",
  "limit": 10,
  "with_payload": true
}
```

Configuring BM25 Parameters

The BM25 ranking function includes three adjustable parameters that you can set at ingest time to optimize search results for your specific use case:

- `k` . Controls term frequency saturation. Higher values increase the influence of term frequency. Defaults to 1.2.
- `b` . Controls document length normalization. Ranges from 0 (no normalization) to 1 (full normalization). A higher value means longer documents have less impact. Defaults to 0.75.
- `avg_len` . Average number of words in the field being queried. Defaults to 256.

For instance, book titles are generally shorter than 256 words. To achieve more accurate scoring when searching for book titles, you could calculate or estimate the average title length and set the `avg_len` parameter accordingly:

```
PUT /collections/books/points?wait=true
{
  "points": [
    {
      "id": 1,
      "vector": {
        "title-bm25": {
          "text": "The Time Machine",
          "model": "qdrant/bm25",
          "options": {
            "avg_len": 5.0
          }
```

```
        }
      },
      "payload": {
        "title": "The Time Machine",
        "author": "H.G. Wells",
        "isbn": "9780553213515"
      }
    }
  ]
}
```

Language-specific Settings

By default, BM25 uses English-specific settings for tokenization, stemming, and stopword removal. Words are reduced to their English root form, and common English stopwords are removed. If your data is not in English, this leads to suboptimal search results. To achieve optimal results for other languages, configure language-specific BM25 settings.

> ⓘ If you set any of the options discussed in this section, ensure that you apply the same settings at ingest and query time.

Stemming and Stopwords

To configure stemming and stopword removal, use the following options:

- `language` : sets the language for stemming and stopword removal. Defaults to `english` . To disable stemming and stopword removal, set `language` to `none` .
- `stemmer` : defaults to stemming for `language` (if set), but can be configured independently.
- `stopwords` : defaults to a set of stopwords for `language` (if set) but can be configured independently. You can configure a specific `language` and/or configure an explicit set of stopwords that will be merged with the stopword set of the configured language.

For example, to use Spanish stemming and stopwords during data ingestion, use:

```
PUT /collections/books/points?wait=true
{
  "points": [
    {
      "id": 1,
      "vector": {
```

```
        "title-bm25": {
          "text": "La Máquina del Tiempo",
          "model": "qdrant/bm25",
          "options": {
            "language": "spanish"
          }
        }
      },
      "payload": {
        "title": "La Máquina del Tiempo",
        "author": "H.G. Wells",
        "isbn": "9788411486880"
      }
    }
  ]
}
```

At query time, use the exact same parameters to ensure consistent text processing:

```
POST /collections/books/points/query
{
  "query": {
    "text": "tiempo",
    "model": "qdrant/bm25",
    "options": {
      "language": "spanish"
    }
  },
  "using": "title-bm25",
  "limit": 10,
  "with_payload": true
}
```

To configure only a stemmer or a stopword set, rather than both, set `language` to `none` and specify the configuration for the desired stemmer or stopwords.

ASCII Folding

ASCII folding is the process of removing diacritics (accents) from characters. By removing diacritics, ASCII folding enables you to ignore accents when searching. For instance, with ASCII folding enabled, searching for “cafe” matches both “cafe” and “café”.

To enable ASCII folding, set the `ascii_folding` option to `true` at both ingest and query time:

```
POST /collections/books/points/query
{
  "query": {
    "text": "Mieville",
    "model": "qdrant/bm25",
    "options": {
      "ascii_folding": true
    }
  },
  "using": "author-bm25",
  "limit": 10,
  "with_payload": true
}
```

Tokenizer

The tokenizer breaks down text into individual tokens (words). By default, the BM25 model uses the `word` tokenizer, which splits text based on word boundaries like whitespace and punctuation. This method is effective for Latin-based languages but may not work well for languages with non-Latin alphabets or languages that do not use spaces to separate words. For those languages, use the `multilingual` tokenizer. This tokenizer supports multiple languages, including those with non-Latin alphabets and non-space delimiters.

```
POST /collections/books/points/query
{
  "query": {
    "text": "村上春樹",
    "model": "qdrant/bm25",
    "options": {
      "tokenizer": "multilingual"
    }
  },
  "using": "author-bm25",
  "limit": 10,
  "with_payload": true
}
```

Language-neutral Text Processing

In some situations, you may want to disable language-specific processing altogether. For example, when searching for author names, that don't necessarily conform to the rules of a specific language.

To disable language-specific processing, set the following options:

- `language`: set to `none` to disable language-specific stemming and stopword removal.
- `tokenizer`: set to `multilingual` for multilingual tokenization and lemmatization.
- Optionally, set `ascii_folding` to `true` to enable ASCII folding and ignore diacritics.

```
POST /collections/books/points/query
{
  "query": {
    "text": "Mieville",
    "model": "qdrant/bm25",
    "options": {
      "language": "none",
      "tokenizer": "multilingual",
      "ascii_folding": true
    }
  },
  "using": "author-bm25",
  "limit": 10,
  "with_payload": true
}
```

SPLADE++

The SPLADE (Sparse Lexical and Dense) family of models are transformer-based models that generate sparse vectors out of text. These models combine the benefits of traditional lexical search with the power of transformer-based models by accounting for homonyms and synonyms. SPLADE models achieve this by expanding the vocabulary of the input text using contextual embeddings from the transformer model. For example, when processing the input text "time travel", a SPLADE model may expand the input to include related terms like "temporal", "journey", and "chronology". This expansion allows SPLADE models to capture the semantic meaning of the text while still leveraging the strengths of lexical search.

The advantage of using SPLADE models is that they perform better than traditional BM25. They also have several downsides though. First, because they use a fixed vocabulary, you

can't use SPLADE models to find terms that are not in the vocabulary, such as product IDs and out-of-domain language (words not seen in training). Secondly, because they are transformer-based models, SPLADE models are slower and require more computational resources than the traditional BM25 model.

On Qdrant Cloud, you can use the SPLADE++ model with inference. Alternatively, you can generate vectors on the client side using the FastEmbed library.

```
POST /collections/books/points/query
{
  "query": {
    "text": "time travel",
    "model": "prithivida/splade_pp_en_v1"
  },
  "using": "title-splade",
  "limit": 10,
  "with_payload": true
}
```

For a tutorial on using SPLADE++ with FastEmbed, refer to How to Generate Sparse Vectors with SPLADE.

miniCOIL

miniCOIL strikes a balance between the flexibility of BM25 and the performance of SPLADE++. miniCOIL is a transformer-based model that generates sparse vectors for text. Unlike SPLADE++, it doesn't use a vocabulary expansion mechanism. To capture the context and meaning of terms, the model generates a four-dimensional vector for each term. miniCOIL does not use a fixed vocabulary, making it an effective model for lexical search that ranks results based on the contextual meaning of keywords.

miniCOIL can be used with the FastEmbed library.

Combining Semantic and Lexical Search with Hybrid Search

Hybrid search enables you to combine semantic and lexical search in a single query, returning results that match the semantic meaning, the exact keywords, or both. This is useful when you don't know whether the user is looking for a specific keyword or a semantically similar document. For example, when searching for books, a user may enter "time travel" to find books related to the concept of time travel, but they may also enter a

book's ISBN to find a specific book. Hybrid queries enable you to return results for both cases in a single query.

Hybrid queries make use of Qdrant's ability to store multiple named vectors in a single point. For example, you can store a dense vector for semantic search and a sparse vector for lexical search in the same point. To do so, first create a collection with both a dense vector and a sparse vector:

```
PUT /collections/books?wait=true
{
  "vectors": {
    "description-dense": {
      "size": 384,
      "distance": "Cosine"
    }
  },
  "sparse_vectors": {
    "isbn-bm25": {
      "modifier": "idf"
    }
  }
}
```

After ingesting data with both vectors, you can use the prefetch feature to run both semantic and lexical queries in a single request. The results of both queries are then combined using a fusion method like Reciprocal Rank Fusion (RRF).

```
POST /collections/books/points/query
{
  "prefetch": [
    {
      "query": {
        "text": "9780553213515",
        "model": "sentence-transformers/all-minilm-l6-v2"
      },
      "using": "description-dense",
      "score_threshold": 0.5
    },
    {
      "query": {
```

```
            "text": "9780553213515",
            "model": "Qdrant/bm25"
          },
          "using": "isbn-bm25"
        }
      ],
      "query": {
        "fusion": "rrf"
      },
      "limit": 10,
      "with_payload": true
    }
```

This query searches for an ISBN, for which only the lexical search returns a result. The `score_threshold` for the semantic query prevents low-scoring results to be returned (0.5 is just an example threshold; you need to tune what a good threshold is for your data and model). So in this case, only the lexical result is returned to the user. If a user had searched for "time travel", only the semantic search would return results, and those would be returned to the user. If a user would search for a term that matched both the semantic and lexical vectors, the results from both searches would be combined to provide a more comprehensive set of results.

You are not limited to prefetching just two queries. Examples include, but are not limited to:

- Fuse multiple lexical queries across the `title`, `author`, and `isbn` fields alongside a semantic query to achieve a comprehensive search across all data.
- Prefetch using sparse or dense vectors and/or filters, and rescore with dense vectors.
- Prefetch with dense and sparse vectors, and rerank using late interaction embeddings.

Conclusion

Qdrant's text search capabilities enable you to build powerful search applications that combine the best of semantic and lexical search. By leveraging dense and sparse vectors, along with Qdrant's flexible querying capabilities, you can create search experiences that meet a wide range of user needs.

Start Free